Implementación de Microservicios Orquestados en AWS

Proyecto del Módulo 7 – MicroPay
Alumno: Macarena Quijada
Fecha: 11/03/2026

---

1. Introducción

Actualmente muchas organizaciones están migrando desde arquitecturas monolíticas hacia arquitecturas basadas en microservicios. Este cambio permite mejorar la escalabilidad, la disponibilidad y la capacidad de actualización de los sistemas.

En el caso de la empresa ficticia **MicroPay**, el sistema actual presenta dificultades para liberar nuevas funcionalidades y experimenta problemas de rendimiento cuando hay alta demanda de usuarios. Para resolver esta situación, se plantea implementar una arquitectura basada en microservicios desplegada en la nube utilizando servicios de AWS.

En este proyecto se desarrollan dos microservicios simples: **Usuarios** y **Pagos**. Cada uno se ejecuta en contenedores Docker, se almacena en Amazon ECR y posteriormente se despliega en Amazon ECS utilizando Fargate. Finalmente, los servicios son accesibles mediante Amazon API Gateway y se monitorean mediante CloudWatch Logs.

Objetivo del proyecto: Comprender de forma práctica cómo desplegar una arquitectura de microservicios en la nube y documentar todo el proceso.

---

2. Desarrollo del proyecto

Lección 1 – Creación de los microservicios

Objetivo: Comprender la separación de funcionalidades en servicios independientes.

Servicios desarrollados:

- **Microservicio Usuarios:** responde a solicitudes relacionadas con la información de usuarios.
- **Microservicio Pagos:** responde a solicitudes relacionadas con el procesamiento de pagos.

Cada microservicio se preparó para ejecutarse dentro de un contenedor mediante Docker. Para ello se creó un archivo Dockerfile que permite construir la imagen del servicio.

Paso a paso visual con nombres y flechas

Usuarios

Cloud9 / Learner Lab

↓ Crear carpeta 'usuarios-service'

↓ Crear archivo 'app.py' (código del microservicio)

↓ Crear Dockerfile

↓ Construir imagen Docker: usuarios-service:1.0

↓ Ejecutar contenedor localmente

↓ Verificar endpoint: http://<URL_CLOUD9>/usuarios

Pagos

Learner Lab

↓ Crear carpeta 'pagos-service'

↓ Crear archivo 'app.py' (código del microservicio)

↓ Crear Dockerfile

↓ Construir imagen Docker: pagos-service:1.0

↓ Ejecutar contenedor localmente

↓ Verificar endpoint: http://abc123xyz.cloud9.us-east-1.amazonaws.com/pagos

Verificación final:

docker images

↓ Confirmar que existen:

usuarios-service:1.0

pagos-service:1.0

Lección 2 – Contenerización y despliegue en AWS

2.1 Creación de repositorios en Amazon ECR

AWS Console → ECR → Create Repository

↓ Crear 'usuarios-service'

↓ Crear 'pagos-service'

↓ Guardar URI de cada repositorio

Captura sugerida: creación de repositorios en ECR

2.2 Subida de imágenes Docker a ECR

docker build -t usuarios-service:1.0 .

↓ docker tag usuarios-service:1.0 123456789012.dkr.ecr.us-east-1.amazonaws.com/usuarios-service:1.0

↓ aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com

↓ docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/usuarios-service:1.0

docker build -t pagos-service:1.0 .

↓docker tag pagos-service:1.0 123456789012.dkr.ecr.us-east-1.amazonaws.com/pagos-service:1.0

↓ docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/pagos-service:1.0

2.3 Creación del cluster ECS

AWS Console → ECS → Create Cluster

↓ Seleccionar Fargate

↓ Asignar nombre: cluster-micropay

↓ Crear cluster

2.4 Creación de Task Definitions

ECS → Task Definitions → Create new Task Definition

↓ Seleccionar Fargate

↓ Configurar contenedor: nombre, imagen ECR, puerto

↓ Crear 'usuarios-task'

↓ Crear 'pagos-task'

Captura sugerida: definición de tarea creada

2.5 Creación de servicios en ECS

ECS → Cluster: cluster-micropay → Create Service

↓ Seleccionar Task Definition

↓ Crear 'usuarios-service'

↓ Crear 'pagos-service'

Lección 3 – Integración con API Gateway

AWS Console → API Gateway → Create HTTP API

↓ Crear rutas:

/usuarios → usuarios-service

/pagos → pagos-service

↓ Publicar API

↓ Obtener URL base: https://abc123defg.execute-api.us-east-1.amazonaws.com

↓ Probar endpoints:

https://abc123defg.execute-api.us-east-1.amazonaws.com/usuarios

https://abc123defg.execute-api.us-east-1.amazonaws.com/pagos

Lección 4 – Monitoreo mediante CloudWatch

AWS Console → CloudWatch → Logs

↓ Revisar grupos de logs de:

usuarios-service

pagos-service

↓ Confirmar solicitudes y respuestas

Capturas sugeridas:

- Logs del microservicio Usuarios
- Logs del microservicio Pagos

Diagrama de arquitectura

Usuarios → API Gateway → ECS Fargate

↓

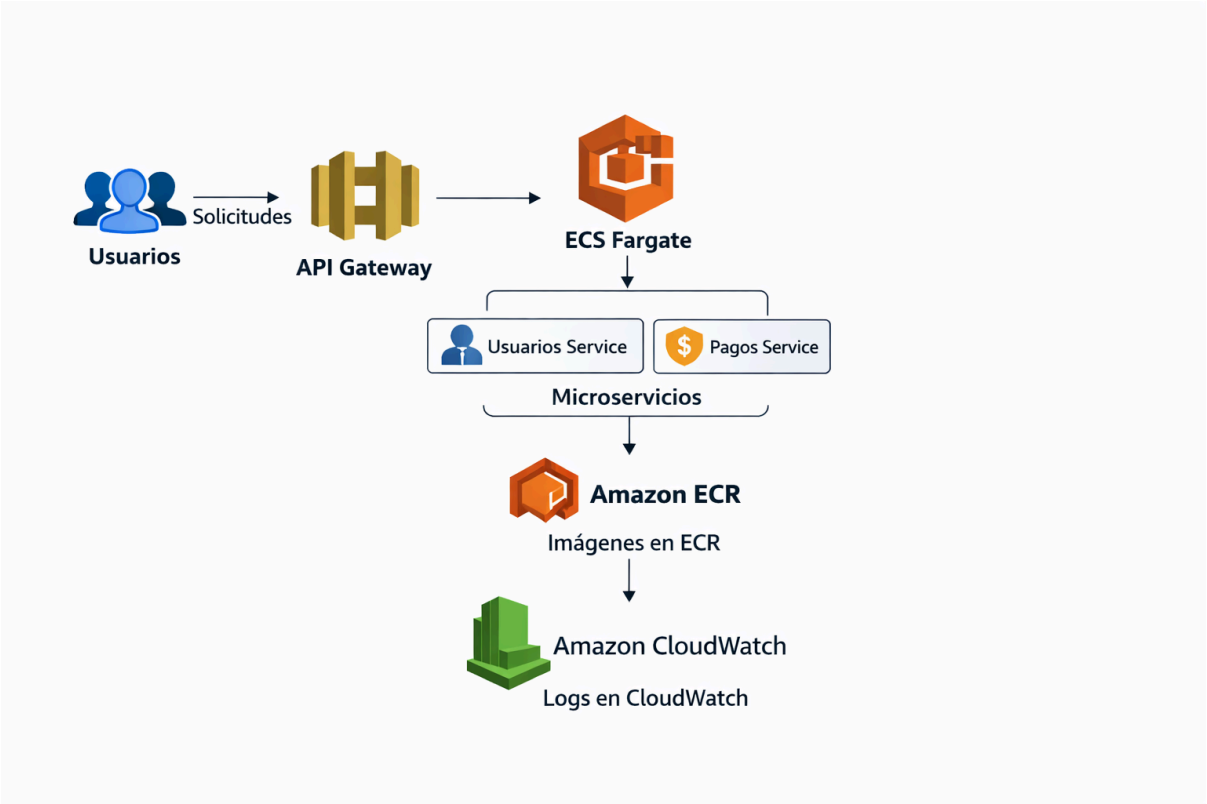
Microservicios

↓

Imágenes en ECR

↓

Logs en CloudWatch



---

3. Conclusiones

- Se comprendió de forma práctica cómo funciona una arquitectura basada en microservicios dentro de un entorno cloud.
- Los contenedores permiten separar funcionalidades en servicios independientes, facilitando su despliegue y mantenimiento.
- Amazon ECS con Fargate simplifica la ejecución de contenedores sin administrar infraestructura.
- API Gateway centraliza el acceso a los servicios, y CloudWatch Logs facilita monitoreo y análisis.
- La arquitectura basada en microservicios ofrece flexibilidad, escalabilidad y eficiencia para aplicaciones modernas.